ChatGPT

InterBrain Obsidian Plugin Implementation

InterBrain is designed as an **interactive knowledge mapping** plugin for Obsidian, focusing on note connections and idea management. It provides a dedicated sidebar panel that combines both outgoing (forward) links and incoming (back) links for the active note (addressing a known Obsidian feature request to unify links in one pane [1]), and additionally suggests related notes through tags, unlinked mentions, and shared connections. The plugin follows Obsidian's API best practices with proper lifecycle hooks, commands, a settings tab, and data storage for user preferences.

Features and Design Overview

- **InterBrain Panel (ItemView):** A sidebar view (opened on the right by default) that displays the **current note's network**:
- **Outgoing Links (Children):** Notes that the current note links to.
- **Incoming Links (Parents):** Notes that link to the current note.
- **Suggested Connections:** Additional related notes, categorized by:
    - *Shared Tags:* Notes sharing one or more tags with the current note.
    - *Unlinked Mentions:* Notes that **mention the current note's title** in their text but are not linked to it (i.e., potential links to create).
    - *Common Links (Siblings):* Notes that share a mutual connection with the current note (e.g. both link to the same third note, or both are linked from the same note).
      The panel updates automatically when you navigate to a different note [2], always reflecting the active note's "knowledge neighborhood." Non-markdown link targets (images, PDFs, etc.) are ignored to focus on note-to-note connections.
- **Commands and Ribbon:** An **"Open InterBrain Panel"** command is added to the command palette, and a brain/network icon on the ribbon for one-click toggling of the panel. Activating it opens (or reveals) the InterBrain view in the sidebar.
- **Settings Tab:** Accessible via Obsidian's settings, allowing users to customize plugin behavior:
- Toggle inclusion of each suggestion category (tags, unlinked mentions, siblings).
- Set the maximum number of suggestions to display per category.
- *(These preferences are saved using Obsidian's* `loadData` / `saveData` *API.)*

Below is the structured implementation of the **InterBrain** plugin. The code is organized into the manifest and three TypeScript modules: the main plugin class, the settings tab, and the view (sidebar panel) class.

Manifest (manifest.json)

This defines the plugin's metadata and ensures it meets Obsidian's requirements. We update the plugin ID, name, description, and entry point. The `main` field points to the compiled main file (`main.js`), and `minAppVersion` is set to Obsidian v1.0.0 to use the latest API features.

```
{
  "id": "interbrain",
  "name": "InterBrain",
  "version": "1.0.0",
  "minAppVersion": "1.0.0",
  "description": "An Obsidian plugin for enhanced note linking and knowledge
mapping, providing an interactive \"brain\" view of connections.",
  "author": "Your Name",
  "authorUrl": "https://your-website.example.com",
  "isDesktopOnly": false,
  "main": "main.js"
}
```

Main Plugin Class (main.ts)

The main plugin class handles loading settings, registering the view, adding commands/ribbon icon, and cleaning up on unload. It uses Obsidian's Plugin API: - **onload():** We load persisted settings (or default values), register the `InterBrainView`, add the settings tab, and create the ribbon icon and command. The view is not opened immediately by default (unless user triggers it). - **onunload():** We detach any open InterBrain view to avoid orphaned leaves. - The `activateView()` method encapsulates opening (or revealing) the InterBrain sidebar using Obsidian's recommended approach for custom views [3] [4].

```
// main.ts
import { App, Plugin, PluginSettingTab, Setting, TFile, WorkspaceLeaf } from
"obsidian";
import { InterBrainSettingTab, InterBrainSettings, DEFAULT_SETTINGS } from "./
settings";
import { InterBrainView, VIEW_TYPE_INTERBRAIN } from "./view";

export default class InterBrainPlugin extends Plugin {
  settings: InterBrainSettings;

  async onload() {
    console.log("Loading InterBrain plugin...");
    // Load settings or fallback to default
    this.settings = Object.assign({}, DEFAULT_SETTINGS, await this.loadData());

    // Register the custom view (ItemView) for the InterBrain panel
    this.registerView(
      VIEW_TYPE_INTERBRAIN,
      (leaf: WorkspaceLeaf) => new InterBrainView(leaf, this.settings)  // pass
settings to the view
    );

    // Add a ribbon icon to toggle the InterBrain panel (uses a network/graph
```

```
icon)
    const ribbonIconEl = this.addRibbonIcon("dot-network", "Open InterBrain
Panel", () => {
      this.activateView();
    });
    // Optionally, add CSS class for styling or active state if needed
    ribbonIconEl.addClass("interbrain-ribbon-icon");

    // Add command to command palette to open the InterBrain panel
    this.addCommand({
      id: "open-interbrain-panel",
      name: "Open InterBrain Panel",
      callback: () => this.activateView()
    });

    // Add the settings tab in Obsidian settings
    this.addSettingTab(new InterBrainSettingTab(this.app, this));
  }

  onunload() {
    console.log("Unloading InterBrain plugin...");
    // Detach any open InterBrain views to clean up
    this.app.workspace.detachLeavesOfType(VIEW_TYPE_INTERBRAIN);
  }

  async saveSettings() {
    await this.saveData(this.settings);
  }

  /** Open (or reveal) the InterBrain view in the right sidebar */
  async activateView() {
    // Close existing leaves of this view type (to avoid duplicates)
    this.app.workspace.detachLeavesOfType(VIEW_TYPE_INTERBRAIN);

    // Create a new leaf in the right sidebar (if false, reuses an existing leaf
if possible)
    const leaf = this.app.workspace.getRightLeaf(false);
    await leaf.setViewState({ type: VIEW_TYPE_INTERBRAIN, active: true });
    // Ensure the new leaf is visible to the user
    this.app.workspace.revealLeaf(leaf);
  }
}
```

Key points in the main class:

- We use `this.registerView` to define our custom view and provide a constructor. The view type `VIEW_TYPE_INTERBRAIN` is a constant string used to identify the view.

- The ribbon icon and command both call `activateView()`, which opens the view. The method follows the pattern of detaching existing leaves of our type, then getting a right sidebar leaf and setting its view state [5]. Finally, `revealLeaf` brings it into focus.
- The settings are loaded via `loadData()` and saved with `saveData()`; `saveSettings()` is called from the settings tab when options change.
- On unload, we ensure any open InterBrain panel is closed (`detachLeavesOfType`) to prevent the UI from retaining a defunct pane.

Settings Module (settings.ts)

The settings module defines the shape of our plugin's settings, default values, and the UI for the settings tab. We provide toggles for including each suggestion category and a slider for the max number of suggestions. These preferences are persisted in the plugin's data file.

```
// settings.ts
import { App, PluginSettingTab, Setting } from "obsidian";
import type InterBrainPlugin from "./
main";  // import the plugin class for type reference

// Define the settings structure
export interface InterBrainSettings {
  includeTagSuggestions: boolean;
  includeMentionSuggestions: boolean;
  includeSiblingSuggestions: boolean;
  maxSuggestionsPerCategory: number;
}

// Default settings values
export const DEFAULT_SETTINGS: InterBrainSettings = {
  includeTagSuggestions: true,
  includeMentionSuggestions: true,
  includeSiblingSuggestions: true,
  maxSuggestionsPerCategory: 5
};

export class InterBrainSettingTab extends PluginSettingTab {
  plugin: InterBrainPlugin;

  constructor(app: App, plugin: InterBrainPlugin) {
    super(app, plugin);
    this.plugin = plugin;
  }

  display(): void {
    const { containerEl } = this;
    containerEl.empty();
```

```
    containerEl.createEl("h3", { text: "InterBrain Plugin Settings" });

    // Toggle: Include Tag-Based Suggestions
    new Setting(containerEl)
      .setName("Include Tag Suggestions")
      .setDesc("Suggest notes that share tags with the current note.")
      .addToggle(toggle =>
        toggle
          .setValue(this.plugin.settings.includeTagSuggestions)
          .onChange(value => {
            this.plugin.settings.includeTagSuggestions = value;
            this.plugin.saveSettings();
          }));

    // Toggle: Include Unlinked Mention Suggestions
    new Setting(containerEl)
      .setName("Include Unlinked Mentions")
      .setDesc("Suggest notes where the current note's title appears in text but
is not linked.")
      .addToggle(toggle =>
        toggle
          .setValue(this.plugin.settings.includeMentionSuggestions)
          .onChange(value => {
            this.plugin.settings.includeMentionSuggestions = value;
            this.plugin.saveSettings();
          }));

    // Toggle: Include "Sibling" Suggestions (common links)
    new Setting(containerEl)
      .setName("Include Mutual Link Suggestions")
      .setDesc("Suggest notes that share a common link or backlink with the
current note.")
      .addToggle(toggle =>
        toggle
          .setValue(this.plugin.settings.includeSiblingSuggestions)
          .onChange(value => {
            this.plugin.settings.includeSiblingSuggestions = value;
            this.plugin.saveSettings();
          }));

    // Slider: Maximum suggestions per category
    new Setting(containerEl)
      .setName("Max Suggestions per Category")
      .setDesc("Limit the number of suggestions shown for each category of
related notes.")
      .addSlider(slider => {
        slider.setLimits(1, 20, 1)
          .setValue(this.plugin.settings.maxSuggestionsPerCategory)
```

```
            .setDynamicTooltip()  // shows the numeric value while sliding
            .onChange(value => {
              this.plugin.settings.maxSuggestionsPerCategory = value;
              this.plugin.saveSettings();
            });
        });
    }
}
```

Highlights:

- We define `InterBrainSettings` and `DEFAULT_SETTINGS` to manage the options.
- `InterBrainSettingTab` extends Obsidian's `PluginSettingTab`. In `display()`, we create UI controls using the fluent `Setting` API.
- Each setting control reflects the current value from `this.plugin.settings` and updates it on change, followed by `this.plugin.saveSettings()` to persist changes.
- We use three toggles (booleans) and one slider (number). For example, **Include Unlinked Mentions** lets users disable scanning for unlinked mentions if they prefer performance over that feature.
- The descriptions clarify each option. The slider is limited from 1 to 20 suggestions per category, with a default of 5 (adjustable as needed).

InterBrain View (Sidebar Panel - view.ts)

This module implements the `ItemView` that renders the InterBrain panel. The view gathers the link data and suggestions for the active note and displays them in a structured way. Key points: - It listens to Obsidian's workspace events (`active-leaf-change` or `file-open`) to update whenever the user switches notes [6]. - It uses Obsidian's Metadata Cache to retrieve forward links and tags, and an internal (undocumented) API to get backlinks [7]. - Suggestions are computed on the fly based on current settings: - **Tag suggestions:** finds notes sharing tags (checking both frontmatter and in-line tags [8]). - **Unlinked mentions:** scans other notes' content for the current note's title (while ignoring already linked instances). - **Mutual link suggestions (siblings):** finds notes that have a common link with the current note (either linking to the same note, or being linked from the same note). - Each suggestion list is capped to the user-defined maximum and excludes notes that are already directly linked to avoid redundancy. - Clicking any note in the panel opens that note in the main workspace.

```
// view.ts
import { ItemView, WorkspaceLeaf, TFile } from "obsidian";
import type { InterBrainSettings } from "./settings";

export const VIEW_TYPE_INTERBRAIN = "INTERBRAIN-VIEW";

export class InterBrainView extends ItemView {
  private settings: InterBrainSettings;

  constructor(leaf: WorkspaceLeaf, settings: InterBrainSettings) {
    super(leaf);
```

```
    this.settings = settings;
  }

  getViewType(): string {
    return VIEW_TYPE_INTERBRAIN;
  }

  getDisplayText(): string {
    return "InterBrain";
  }

  getIcon(): string {
    // Use a built-in icon (network graph icon) for the view tab
    return "dot-network";
  }

  async onOpen(): Promise<void> {
    // When the view opens, set up event listener to update on file changes
    this.registerEvent(
      // Use the 'file-open' event to update when active file changes
      this.app.workspace.on("file-open", (file: TFile | null) => {
        this.renderForFile(file);
      })
    );
    // Initial render for the currently active file (if any)
    this.renderForFile(this.app.workspace.getActiveFile());
  }

  onClose(): Promise<void> {
    // Nothing special to clean up (events are auto-unregistered on close)
    return Promise.resolve();
  }

  /** Render the InterBrain panel content for a given file (or show a message if
null). */
  private async renderForFile(file: TFile | null) {
    const { contentEl } = this;
    contentEl.empty();  // Clear previous content

    if (!file) {
      contentEl.createEl("div", { text: "No note is open.", cls: "interbrain-
none" });
      return;
    }

    // Display the header with the current note title
    const noteTitle = file.basename;
    contentEl.createEl("h3", { text: `InterBrain: ${noteTitle}` });
```

```
    // Gather forward (outgoing) links from this note
    const forwardLinks: Record<string, number> =
this.app.metadataCache.resolvedLinks[file.path] || {};
    const forwardPaths = Object.keys(forwardLinks).filter(p => p);  // keys are
target paths
    // Gather backlinks (incoming links) to this note (undocumented API) 7
    let backPaths: string[] = [];
    const backlinks = (this.app.metadataCache as any).getBacklinksForFile(file);
    if (backlinks && backlinks.data) {
      backPaths = Object.keys(backlinks.data);
    }

    // Filter out non-markdown files from forward/back links to focus on notes
    const forwardNotePaths = forwardPaths.filter(path => path.endsWith(".md"));
    const backNotePaths = backPaths.filter(path => path.endsWith(".md"));

    // Remove self-references if any (a note linking to itself, uncommon)
    const currPath = file.path;
    const forwardSet = new Set(forwardNotePaths.filter(p => p !== currPath));
    const backSet = new Set(backNotePaths.filter(p => p !== currPath));
    const directLinkSet = new Set<string>([...forwardSet, ...backSet]);

    // ** Outgoing Links Section **
    contentEl.createEl("h4", { text: "Linked notes (outgoing):" });
    if (forwardSet.size > 0) {
      const list = contentEl.createEl("ul");
      forwardSet.forEach(path => {
        const targetFile = this.app.vault.getAbstractFileByPath(path);
        if (targetFile && targetFile instanceof TFile && targetFile.extension
=== "md") {
          const item = list.createEl("li");
          const link = item.createEl("a", { text: targetFile.basename, href:
"#" });
          link.onclick = (ev: MouseEvent) => {
            ev.preventDefault();
            this.openFile(targetFile as TFile);
          };
        }
      });
    } else {
      contentEl.createEl("div", { text: "None", cls: "interbrain-none" });
    }

    // ** Incoming Links Section **
    contentEl.createEl("h4", { text: "Linked from (backlinks):" });
    if (backSet.size > 0) {
      const list = contentEl.createEl("ul");
```

```
      backSet.forEach(path => {
        const srcFile = this.app.vault.getAbstractFileByPath(path);
        if (srcFile && srcFile instanceof TFile && srcFile.extension === "md") {
          const item = list.createEl("li");
          const link = item.createEl("a", { text: srcFile.basename, href:
"#" });
          link.onclick = (ev: MouseEvent) => {
            ev.preventDefault();
            this.openFile(srcFile as TFile);
          };
        }
      });
    } else {
      contentEl.createEl("div", { text: "None", cls: "interbrain-none" });
    }

    // ** Suggestions Section ** (if enabled in settings)
    // Prepare a cap for suggestions
    const maxSuggest = this.settings.maxSuggestionsPerCategory;

    // (A) Tag-based suggestions
    if (this.settings.includeTagSuggestions) {
      contentEl.createEl("h4", { text: "Notes with Shared Tags:" });
      // Collect current note's tags (from frontmatter and in-body tags) 8
      const thisCache = this.app.metadataCache.getFileCache(file);
      const currTags = new Set<string>();
      if (thisCache) {
        // Frontmatter tags
        const fmTags = thisCache.frontmatter?.tags;
        if (fmTags) {
          if (Array.isArray(fmTags)) {
            fmTags.forEach(tag => {
              if (typeof tag === "string") currTags.add(tag.replace(/^#/,
"").toLowerCase());
            });
          } else if (typeof fmTags === "string") {
            fmTags.split(/[\s,]+/).forEach(tag => {
              if (tag) currTags.add(tag.replace(/^#/, "").toLowerCase());
            });
          }
        }
        // In-document #tags
        if (thisCache.tags) {
          thisCache.tags.forEach((t: any) => {
            if (t.tag) currTags.add(t.tag.replace(/^#/, "").toLowerCase());
          });
        }
      }
```

```
      if (currTags.size === 0) {
        // No tags in current note
        contentEl.createEl("div", { text: "None (no tags in this note)", cls:
"interbrain-none" });
      } else {
        // Iterate all markdown files to find shared tags
        const tagSuggestions: { file: TFile, count: number }[] = [];
        const allNotes = this.app.vault.getMarkdownFiles();
        for (const otherFile of allNotes) {
          if (otherFile.path === currPath) continue;  // skip self
          // Skip if already directly linked (already in network)
          if (directLinkSet.has(otherFile.path)) continue;
          const otherCache = this.app.metadataCache.getFileCache(otherFile);
          if (!otherCache) continue;
          // Collect tags of the other file
          const otherTags = new Set<string>();
          const fmTags2 = otherCache.frontmatter?.tags;
          if (fmTags2) {
            if (Array.isArray(fmTags2)) {
              fmTags2.forEach((tag: any) => {
                if (typeof tag === "string") otherTags.add(tag.replace(/^#/,
"").toLowerCase());
              });
            } else if (typeof fmTags2 === "string") {
              fmTags2.split(/[\s,]+/).forEach(tag => {
                if (tag) otherTags.add(tag.replace(/^#/, "").toLowerCase());
              });
            }
          }
          if (otherCache.tags) {
            otherCache.tags.forEach((t: any) => {
              if (t.tag) otherTags.add(t.tag.replace(/^#/, "").toLowerCase());
            });
          }
          // Count common tags
          let commonCount = 0;
          currTags.forEach(tag => {
            if (otherTags.has(tag)) commonCount++;
          });
          if (commonCount > 0) {
            tagSuggestions.push({ file: otherFile, count: commonCount });
          }
        }
        if (tagSuggestions.length === 0) {
          contentEl.createEl("div", { text: "None", cls: "interbrain-none" });
        } else {
          // Sort suggestions by number of common tags (descending), then
alphabetically
```

```
          tagSuggestions.sort((a, b) => b.count - a.count ||
a.file.basename.localeCompare(b.file.basename));
          const list = contentEl.createEl("ul");
          for (let i = 0; i < tagSuggestions.length && i < maxSuggest; i++) {
            const { file: sugFile, count } = tagSuggestions[i];
            const item = list.createEl("li");
            const link = item.createEl("a", { text: sugFile.basename, href:
"#" });
            // Tooltip shows how many tags in common
            link.setAttr("title", `Shares ${count} tag(s) in common`);
            link.onclick = (ev: MouseEvent) => {
              ev.preventDefault();
              this.openFile(sugFile);
            };
          }
          if (tagSuggestions.length > maxSuggest) {
            list.createEl("li", { text: `…and ${tagSuggestions.length -
maxSuggest} more`, cls: "interbrain-more" });
          }
        }
      }
    }

    // (B) Unlinked mention suggestions
    if (this.settings.includeMentionSuggestions) {
      contentEl.createEl("h4", { text: `Notes mentioning "${noteTitle}":` });
      const titleLower = noteTitle.toLowerCase();
      const mentionSuggestions: { file: TFile, count: number }[] = [];
      const allNotes = this.app.vault.getMarkdownFiles();
      for (const otherFile of allNotes) {
        if (otherFile.path === currPath) continue;
        if (directLinkSet.has(otherFile.path)) continue;
        // Read the file content (cached read)
        const text = await this.app.vault.cachedRead(otherFile);
        if (!text) continue;
        const textLower = text.toLowerCase();
        // Skip if the current note is already linked in this file (presence of
a wiki-link)
        // e.g., if "[[NoteTitle]]" exists, then it's not an unlinked mention
        if (textLower.includes("[[" + titleLower)) {
          continue;
        }
        // Check for raw title occurrences in text (word-boundary search)
        const regex = new RegExp(`\\b${this.escapeRegExp(noteTitle)}\\b`, "gi");
        const matches = text.match(regex);
        if (matches && matches.length > 0) {
          mentionSuggestions.push({ file: otherFile, count: matches.length });
        }
```

```
      }
      if (mentionSuggestions.length === 0) {
        contentEl.createEl("div", { text: "None", cls: "interbrain-none" });
      } else {
        // Sort by frequency of mention (descending), then name
        mentionSuggestions.sort((a, b) => b.count - a.count ||
a.file.basename.localeCompare(b.file.basename));
        const list = contentEl.createEl("ul");
        for (let i = 0; i < mentionSuggestions.length && i < maxSuggest; i++) {
          const { file: sugFile, count } = mentionSuggestions[i];
          const item = list.createEl("li");
          const link = item.createEl("a", { text: sugFile.basename, href:
"#" });
          link.setAttr("title", `Mentions "${noteTitle}" ${count} time(s)`);
          link.onclick = (ev: MouseEvent) => {
            ev.preventDefault();
            this.openFile(sugFile);
          };
        }
        if (mentionSuggestions.length > maxSuggest) {
          list.createEl("li", { text: `…and ${mentionSuggestions.length -
maxSuggest} more`, cls: "interbrain-more" });
        }
      }
    }

    // (C) Mutual connection ("sibling") suggestions
    if (this.settings.includeSiblingSuggestions) {
      contentEl.createEl("h4", { text: "Notes with Mutual Connections:" });
      const siblingMap: Record<string, number> = {};
      // Siblings via common parent (share an incoming link)
      backSet.forEach(parentPath => {
        // For each note that links to current, get its outgoing links
        const outMap: Record<string, number> =
this.app.metadataCache.resolvedLinks[parentPath] || {};
        for (const targetPath in outMap) {
          if (!targetPath.endsWith(".md")) continue;
          if (targetPath === currPath) continue;
          if (directLinkSet.has(targetPath)) continue;
          siblingMap[targetPath] = (siblingMap[targetPath] || 0) + 1;
        }
      });
      // Siblings via common child (share an outgoing link)
      forwardSet.forEach(childPath => {
        const childFile = this.app.vault.getAbstractFileByPath(childPath);
        if (!childFile || !(childFile instanceof TFile)) return;
        const childBacklinks = (this.app.metadataCache as
any).getBacklinksForFile(childFile);
```

```
        if (childBacklinks && childBacklinks.data) {
          for (const parentPath in childBacklinks.data) {
            if (!parentPath.endsWith(".md")) continue;
            if (parentPath === currPath) continue;
            if (directLinkSet.has(parentPath)) continue;
            siblingMap[parentPath] = (siblingMap[parentPath] || 0) + 1;
          }
        }
      });
      const siblingSuggestions: { file: TFile, count: number }[] = [];
      for (const path in siblingMap) {
        const count = siblingMap[path];
        const fileObj = this.app.vault.getAbstractFileByPath(path);
        if (fileObj && fileObj instanceof TFile && fileObj.extension === "md") {
          siblingSuggestions.push({ file: fileObj, count });
        }
      }
      if (siblingSuggestions.length === 0) {
        contentEl.createEl("div", { text: "None", cls: "interbrain-none" });
      } else {
        // Sort by number of mutual connections (desc), then name
        siblingSuggestions.sort((a, b) => b.count - a.count ||
a.file.basename.localeCompare(b.file.basename));
        const list = contentEl.createEl("ul");
        for (let i = 0; i < siblingSuggestions.length && i < maxSuggest; i++) {
          const { file: sugFile, count } = siblingSuggestions[i];
          const item = list.createEl("li");
          const link = item.createEl("a", { text: sugFile.basename, href:
"#" });
          link.setAttr("title", `Shares ${count} mutual connection(s)`);
          link.onclick = (ev: MouseEvent) => {
            ev.preventDefault();
            this.openFile(sugFile);
          };
        }
        if (siblingSuggestions.length > maxSuggest) {
          list.createEl("li", { text: `…and ${siblingSuggestions.length -
maxSuggest} more`, cls: "interbrain-more" });
        }
      }
    }
  }

  /** Open a file in the main workspace (without affecting the InterBrain view)
*/
  private openFile(file: TFile) {
    // Open the given file in a new tab (or existing leaf if available)
    this.app.workspace.getLeaf(true).openFile(file);
```

```
  }

  /** Escape RegExp special characters in a string (for safe regex building) */
  private escapeRegExp(str: string): string {
    return str.replace(/[-\/\\^$*+?.()|[\]{}]/g, "\\$&");
  }
}
```

Explanation of the view logic:

- **Event Handling:** In `onOpen()`, we register an event handler for `'file-open'` events [6]. Whenever the user opens or switches to a different note, `file-open` fires and we call `renderForFile(file)`. We also call it immediately for the current active file to initialize the view. The use of `this.registerEvent` ties the event's lifecycle to the view (it will auto-unregister on view close), so we don't leak listeners.
- **Rendering Links:** We use Obsidian's `metadataCache.resolvedLinks` to get outgoing links (this gives a map of `targetPath -> count` for each link from the current file). For incoming links, we call the internal `getBacklinksForFile` (not officially in the API typings [7]) which returns an object with a `.data` property mapping each source file path to its links. We extract the keys of `.data` to get all file paths that mention the current note. These operations are efficient since Obsidian indexes links internally (though `getBacklinksForFile` internally iterates the cache [9], it is fast enough for typical use). We filter out non-markdown files from these lists, as we only want notes.
- Outgoing and incoming links are displayed under **"Linked notes (outgoing):"** and **"Linked from (backlinks):"** respectively. Each entry is an `<a>` link; clicking it opens that note using `this.openFile()`. We skip listing the current note itself if it somehow appears (to avoid self-loop).
- **Tag Suggestions:** If enabled, we collect the current note's tags from both frontmatter and in-line tags [8]. We then iterate through all markdown files in the vault (via `app.vault.getMarkdownFiles()` which returns all notes) and compute how many tags each shares with the current note. Any note with at least one common tag (and which is not already directly linked to the current note) is a candidate. We then sort these suggestions by the number of shared tags (descending) and take the top *N* (user-configured). Each suggested note shows as a list item, and we set a tooltip indicating the number of common tags. If no note shares a tag, we display "None". (If the current note has no tags at all, we note that and skip scanning.)
- **Unlinked Mentions:** If enabled, we search for the current note's title appearing in other notes' bodies without an existing link. We read each note's content (`vault.cachedRead`) and skip it if it contains a wiki link to the current note (we check for `[[title` in a case-insensitive way as a quick heuristic; if found, we assume the note is already linked and ignore it). Then we use a regex with word boundaries (e.g., `\bTitle\b`) to count raw text mentions of the title [8]. This helps ensure we match full words/phrases rather than substrings of other words. Each note with one or more unlinked mentions is listed with a tooltip indicating how many times the title appears. We again sort by frequency and cap the list to *N*. If none are found, we show "None". (This feature can be computationally expensive for large vaults, so it can be turned off via settings if desired.)
- **Mutual Connections (Siblings):** If enabled, we look for notes that are one step away through a shared connection:

- *Via common parent:* For each note that links **into** the current note (each "parent" in the backlinks list), we take all of its outgoing links. Any note that a parent links to (excluding the current note and any already directly linked notes) is a **sibling candidate** (the parent is a common connection between the current note and that candidate).
- *Via common child:* For each note that the current note links **out to** (each "child" in the forward links list), we find all notes that link to that child. Any such note (excluding the current note and direct links) is also a sibling candidate (they share the child as a mutual connection). We retrieve these via `getBacklinksForFile` on the child note.
- We tally these occurrences in `siblingMap` so if a note is related through multiple mutual connections, it gets a higher count. For example, if Note X and the current note share two different common links, X's count will be 2 (indicating a stronger connection).
- Finally, we convert `siblingMap` to a list of notes with counts, sort by count (desc), and list the top *N*. The tooltip shows how many mutual connections were found (e.g., "shares 2 mutual connection(s)"). If none, we display "None".
- **Opening Notes:** The `openFile` helper uses Obsidian's workspace API to open the given file in a new leaf (tab) in the main area. We use `getLeaf(true)` which opens a new tab if possible [10], so we don't disturb the current note open in the main view. This allows users to click through multiple suggestions without closing the InterBrain panel or the original note.
- **Utility:** We include an `escapeRegExp` function to safely escape note titles for regex usage (important if the title contains special regex characters like `*` or `+`). This prevents errors in the mention search.

Each section in the panel is clearly labeled and only shown if relevant (or explicitly stating "None" if no results, so the user knows the check was performed). By combining immediate links and indirect connections, the **InterBrain** panel serves as a hub for exploring relationships around a note, living up to the "brain" metaphor of a network of knowledge.

Usage: After installing the plugin, use the **InterBrain ribbon icon** (a network node graphic on the left toolbar) or the **"Open InterBrain Panel"** command from the palette to toggle the sidebar. Click any note in the panel to open it. Adjust the plugin settings to include or exclude suggestion types and set the desired number of suggestions. All settings are persisted across Obsidian sessions.

This completes the implementation of the InterBrain plugin, which transforms the initial skeleton into a fully functional feature-rich plugin. The code adheres to Obsidian's API standards and best practices, ensuring a stable integration into the Obsidian environment. Enjoy mapping your notes with InterBrain! [1] [7]

---

[1] How to get backlinks for a file? - Developers: Plugin & API - Obsidian Forum
https://forum.obsidian.md/t/how-to-get-backlinks-for-a-file/45314

[2] [7] [8] Getting backlinks, tags and frontmatter entries for a note - Developers: Plugin & API - Obsidian Forum
https://forum.obsidian.md/t/getting-backlinks-tags-and-frontmatter-entries-for-a-note/34082

[3] [4] [5] How to correctly open an ItemView? - Developers: Plugin & API - Obsidian Forum
https://forum.obsidian.md/t/how-to-correctly-open-an-itemview/60871

6 10 Workspace - Developer Documentation
https://docs.obsidian.md/Reference/TypeScript+API/Workspace

9 Store backlinks in metadataCache - Developers: Plugin & API - Obsidian Forum
https://forum.obsidian.md/t/store-backlinks-in-metadatacache/67000